Computer Communications and Networks



Lab 02
Introduction to Wireshark
Packet Capture and Protocol Header Analysis Using Wireshark

Lab Manual 02

Objectives

- Introduction to Wireshark
- Running Wireshark
- Explore Wireshark Filters
- Use Wireshark to capture network packets.
- Recognize various protocol headers such as Ethernet, IP, and TCP/UDP.
- Inspect and analyze the Hex dump of captured packets, identifying key header field values.

Reference Material

Introduction to Wireshark

Wireshark, a network analysis tool formerly known as Ethereal, captures packets in real time and display them in human-readable format. Wireshark includes filters, color-coding and other features that let you dig deep into network traffic and inspect individual packets.

Running Wireshark

When you run the Wireshark program, the Wireshark graphical user interface shown in Figure 1 will displayed. Initially, no data will be displayed in the various windows.

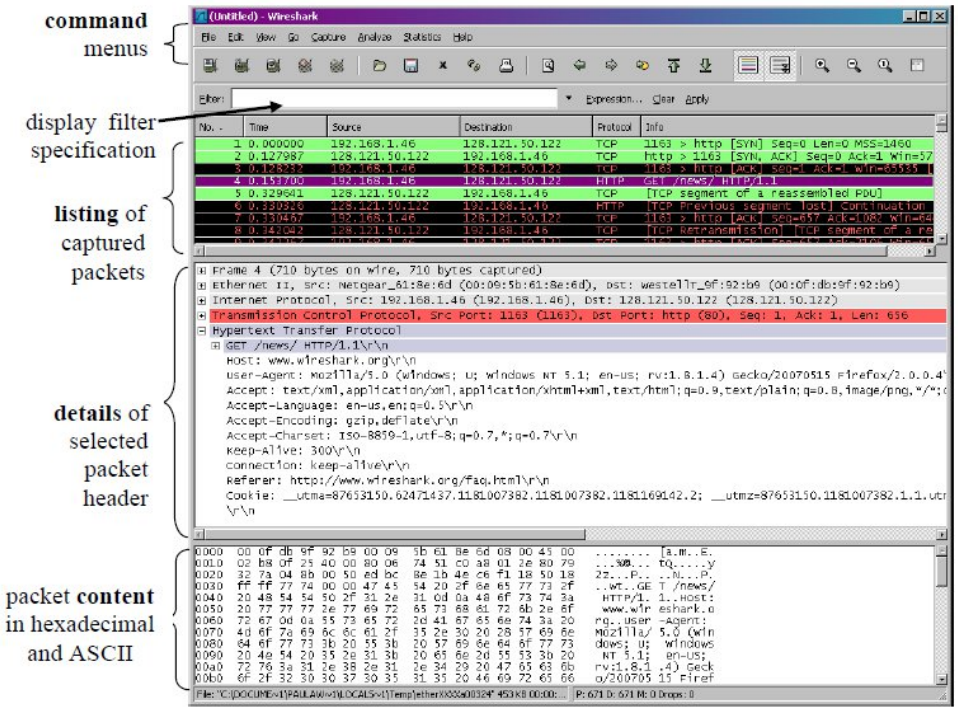


Figure: Wireshark Graphical User Interface

Wireshark Filters

The simplest filter allows you to check for the existence of a protocol or field. If you want to see all packets which contain the IP protocol, the filter would be "ip" (without the quotation marks).

1. Comparison operators

Fields can also be compared against values. The comparison operators can be expressed either through English-like abbreviations or through C-like symbols:

eq, == Equal
ne, != Not Equal
gt, > Greater Than
lt, < Less Than
ge, >= Greater than or Equal to
le, <= Less than or Equal to

Example

ip.addr == 10.0.0.1 [Sets a filter for any packet with 10.0.0.1, as either the source or dest]
tcp.time_delta > .250 [sets a filter to display all tcp packets that have a delta time of greater than 250mSec in the context of their stream

1. Search and match operators

Additional operators exist expressed only in English, not C-like syntax:

contains Does the protocol, field contain a value
matches, ~ Does the protocol or text string match the given case-insensitive Perl-compatible regular expression

The "contains" operator allows a filter to search for a sequence of characters, expressed as a string (quoted or unquoted), or bytes, expressed as a byte array, or for a single character, expressed as a C-style character constant.

Example

To search for a given HTTP URL in a capture, the following filter can be used:

http contains https://www.wireshark.org

The "contains" operator cannot be used on atomic fields, such as numbers or IP addresses.

The "matches" or "~" operator allows a filter to apply to a specified Perl-compatible regular expression (PCRE). The "matches" operator is only implemented for protocols and for protocol fields with a text string representation. Matches are case-insensitive by default. For example, to search for a given WAP WSP User-Agent, you can write:

wsp.user_agent matches "cldc"

This would match "cldc", "CLDC", "cLdC" or any other combination of upper and lower case letters.

2. The membership operator

A field may be checked for matches against a set of values simply with the membership operator. For instance, you may find traffic on common HTTP/HTTPS ports with the following filter:

tcp.port in {80 443 8080}

as opposed to the more verbose:

tcp.port == 80 or tcp.port == 443 or tcp.port == 8080

Example

To find HTTP requests using the HEAD or GET methods:

http.request.method in {"HEAD" "GET"}

The set of values can also contain ranges:

tcp.port in {443 4430..4434}

ip.addr in {10.0.0.5 .. 10.0.0.9 192.168.1.1..192.168.1.9}

frame.time_delta in {10 .. 10.5}

3. Logical expressions

Tests can be combined using logical expressions. These too are expressible in C-like syntax or with English-like abbreviations:

and, && Logical AND

or, || Logical OR

not, ! Logical NOT

Example

ip.addr==10.0.0.1 && ip.addr==10.0.0.2 [sets a conversation filter between the two defined IP addresses]

!(arp or icmp or stp) [masks out arp, icmp, stp, or whatever other protocols may be background noise.

Expressions can be grouped by parentheses as well. The following are all valid display filter expressions:

tcp.port == 80 and ip.src == 192.168.2.1

not llc

http and frame[100-199] contains "wireshark"

(ipx.src.net == 0xbad && ipx.src.node == 0.0.0.0.0.1) || ip

Taking Wireshark for a Test Run:

- **Capturing Packets Using Wireshark:**
  - Launch Wireshark and select an appropriate network interface (e.g., Ethernet or Wi-Fi).
  - Click the 'Start Capture' button to begin capturing live network traffic.
  - While the capture is running, perform network activity such as browsing the web or sending a file.
  - After a few minutes of activity, stop the capture.

- **Applying Filters and Analyzing Packets:**
    - Use the display filter `ip` to focus on IP traffic. Explore other filters such as `tcp`, `udp`, and `http` to analyze specific types of packets.
    - Select a packet from the filtered list and inspect the various protocol headers (Ethernet, IP, and TCP/UDP) in the packet detail window.

- **Inspecting the Hex Dump:**
    - In the Wireshark interface, select a packet and open its "Hex dump" view.
    - Identify and analyze the key fields in the Ethernet, IP, and TCP/UDP headers by examining their hexadecimal values.
    - For example, in an IP packet:
        - **Source IP address:** Located in bytes 12–15.
        - **Destination IP address:** Located in bytes 16–19.
    - For TCP packets, examine fields like:
        - **Source Port:** First two bytes of the TCP header.
        - **Destination Port:** Next two bytes of the TCP header.

Lab Tasks

Explore the packets you captured from test run and answer the following question

- What are the protocols involved in the captured packets?
- Identify and list the source and destination IP addresses from the packet headers.
- From the Hex dump, what is the value of the "Protocol" field in the IP header? What protocol does this value represent?
- Analyze the TCP segment and note the source and destination port numbers. What services do these ports represent?
- Inspect the Ethernet frame. What are the source and destination MAC addresses?

"Learn like TCP—reliable, ordered, and persistent; play like UDP—curious and quick. If life drops a packet, fire up Wireshark, retransmit, and keep the connection alive!"

WireShark Quest

Task 1 — Quick Capture & Interface

Open Wireshark and select your active interface (Wi-Fi/Ethernet).

Start a 2-minute capture while you browse any safe website.

Stop and apply the display filter: ip

📸 Screenshot: packet list + filter bar visible.

Task 2 — Display Filters Basics

Show only TCP or UDP traffic using: tcp or udp

Find any HTTP or HTTPS packet using: http or tcp.port in {80 443}

Expand the Internet and Transport header fields in the middle pane.

📸 Screenshot: highlight Source/Destination ports and IPs.

Task 3 — Conversations & Streams

Right-click a TCP packet → Follow → TCP Stream to view the flow.

Use Statistics → Conversations → TCP to identify top talkers.

Apply stream filter (e.g., tcp.stream == X) to isolate one flow.

📸 Screenshot: Follow Stream window + one filtered stream in main view.

Task 4 — Timing & Performance

Apply: tcp.time_delta > 0.250 to locate delayed packets.

Enable the "Time delta from previous displayed frame" column (Right-click a column header → Column Preferences → +).

Identify any spike and note which conversation caused it.

📸 Screenshot: packet with large delta highlighted + column visible.

Task 5 — Protocol Details & Hex

Select one IP packet and open the Hex view.

Identify bytes 12–15 (Source IP) and 16–19 (Destination IP).

Find the IP Protocol field value and map it to its protocol (e.g., 6 = TCP).

📸 Screenshot: Hex view with the relevant bytes highlighted.

Task 6 — Custom Color Rule & Handshake

Create a color rule that highlights TCP SYN packets (no ACK).

Filter SYNs using: tcp.flags.syn == 1 and tcp.flags.ack == 0

Identify a three-way handshake (SYN → SYN,ACK → ACK).

Estimate RTT by comparing timestamps between SYN and SYN,ACK.

📸 Screenshot: at least one handshake with your color rule visible.